



STUDENT HOSTEL MANAGEMENT SYSTEM

PROJECT REPORT

Academic Year: 2026–2027

Project Title: Hostel Management System

Technology Used: Java, Java Swing, MySQL

Project Type: Java Mini Project / Desktop Application

Submitted By:

Name: Roopesh Singh

Registration No.: 25BAI11006

Course: B.Tech CSE (AI & ML)

1. INTRODUCTION

Keeping student hostel records is a task in every college or university. Information such as student names, roll numbers, courses, phone numbers and room numbers must be kept carefully so that they can be found whenever needed. When these records are handled by hand finding or updating details can take a time.

The Student Hostel Management System is built as a desktop application to make this process more organized and convenient. The project uses Java Swing for the graphical user interface JDBC for connecting to the database and MySQL for storing student records.

The application lets the user add student details view stored records, update existing information, delete records and search for students using various details. The project follows a design where the user interface,

data model, database operations and connection handling are kept in separate classes.

Overall this project shows how Java programming, GUI development, SQL queries and database connectivity work together in a real-world record management application.

2. PROBLEM STATEMENT

Keeping student hostel records by hand can become hard as the number of students grows. Information such as names, roll numbers, courses, phone numbers and room numbers might be stored in paper registers or separate files making it slow to find and update details.

Handwritten record keeping can also lead to entries, duplicate records and trouble keeping information up to date. Searching for a student may require checking many pages

or files which adds to the workload of the hostel staff.

Therefore there is a need for a computerized system that can store student information in one place and provide basic operations such as adding, viewing, updating, deleting and searching records.

The Student Hostel Management System aims to solve these problems by offering an easy-to-use desktop application for managing hostel student information.

3. OBJECTIVES

The main objectives of the project are:

- To develop a desktop application for managing hostel student records.**
- To reduce the dependency on record-keeping methods.**
- To store student details in a database.**

- **To provide an interface for adding new student records.**
 - **To allow users to view all available student information.**
 - **To provide options for updating incorrect or outdated details.**
 - **To allow deletion of records when they are no longer required.**
 - **To make student searching faster and more convenient.**
 - **To understand the practical implementation of Java Swing and JDBC.**
 - **To develop a modular application that can be improved in the future.**
-

4. SCOPE OF THE PROJECT

The scope of this project focuses on the management of student hostel records. The application is designed to handle student information such as:

- **Student name**
- **Roll number**
- **Course**
- **year**
- **Phone number**
- **Room number**

The current system supports the following operations:

- **Adding student records**
- **Viewing all student records**
- **Updating existing records**
- **Deleting student records**
- **Searching student information**
- **Clearing form fields**
- **Refreshing the displayed records**

The project is mainly meant for small-scale hostel record management. It does not yet include features such, as online payments,

attendance tracking, student login, automated room allocation or complaint management. These features can be added in development.

5. TARGET USERS

The main target users of this application are:

Hostel Administrators

Hostel administrators can use the system to maintain student records and update information whenever required.

Hostel Office Staff

Office staff can use the application to search student details verify records and manage hostel-related information.

College Management

College management can use the stored information for record verification and administrative purposes.

Project Demonstration Users

The application can also be used by students or faculty members for demonstrating Java GUI database connectivity and CRUD operations.

The system is mainly designed for authorized users who're responsible for maintaining hostel records.

6. FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that the system should perform.

6.1 Add Student

The user should be able to enter student details such as name, roll number, course, year, phone number and room number. The entered information should be stored in the database.

6.2 View Student Records

The system should display the student records in a table format. The table includes details such as ID, name, roll number, course, year, phone number and room number.

6.3 Update Student Details

The user should be able to select an existing student record and update incorrect or changed information.

6.4 Delete Student Record

The system should allow the user to delete a selected student record from the database.

6.5 Search Student

The user should be able to search for student records using details such as:

- Student name**
- Roll number**
- Room number**

6.6 Clear Form

The Clear button should remove the entered values from the input fields so that the user can enter information.

6.7 Refresh Records

The Refresh option should reload the records from the database and display them in the table.

6.8 Database Connectivity

The application should connect with the MySQL database using JDBC and execute SQL queries for storing and retrieving information.

7. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and performance expected from the system.

7.1 Usability

The application should have an understandable interface so that users can

operate it without requiring advanced technical knowledge.

7.2 Performance

The system should perform operations such as adding, searching and updating records within a reasonable amount of time.

7.3 Reliability

The application should correctly. Retrieve student information without unnecessary data loss.

7.4 Maintainability

The code should be divided into classes so that changes can be made easily in the future.

7.5 Scalability

The system should be capable of handling an increasing number of student records without changes in its basic structure.

7.6 Security

Database credentials and connection details should be handled carefully. Authorized users should be allowed to manage hostel records.

7.7 Portability

The application can run on systems that have Java installed and the required database connectivity setup available.

8. SYSTEM ARCHITECTURE

The project follows a layered or modular architecture. Different responsibilities are divided into classes to make the application easier to understand and maintain.

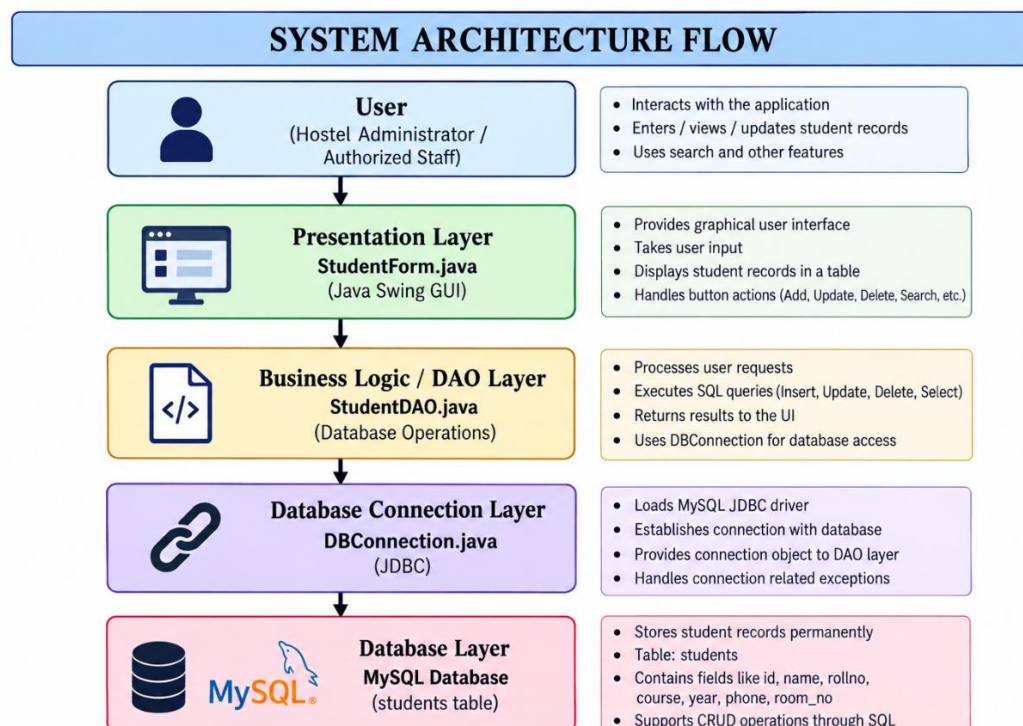


Figure 1: System Architecture Flow

The main components of the system are:

8.1 User Interface Layer

The user interface is developed using Java Swing. It contains text fields, buttons, tables and other components through which the user interacts with the application.

Main class:

- **StudentForm.java**

8.2 Model Layer

The model class represents the student data used by the application. It stores information related to a student.

Main class:

- **Student.java**

8.3 Data Access Layer

The DAO layer is responsible for executing SQL queries and performing database operations such, as insert, update, delete and select.

Main class:

- **StudentDAO.java**

8.4 Database Connection Layer

This layer manages the connection between the Java application and the MySQL database using JDBC.

Main class:

- **DBConnection.java**

8.5 Database Layer

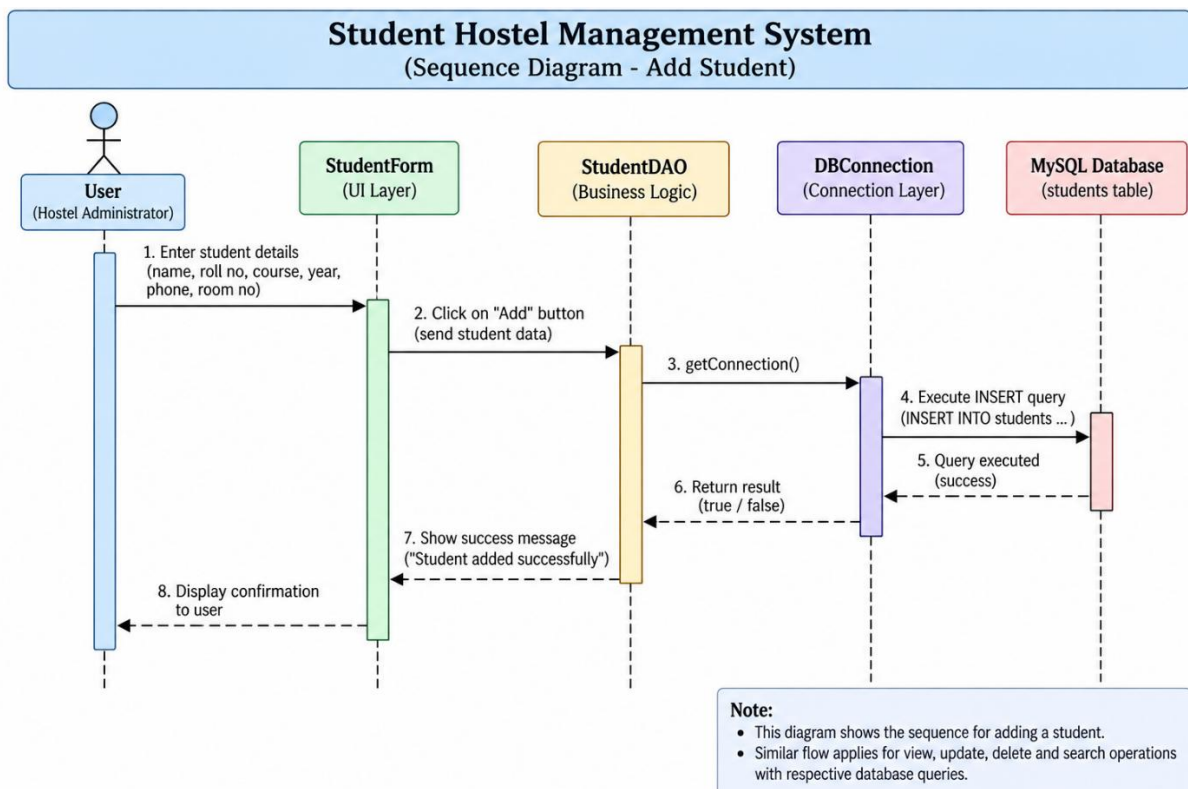
MySQL is used to store student records. The database holds a table called students.

Architecture Flow

This structure keeps the interface code and the database code separate, which makes the project more organized.

9. DESIGN ARTIFACTS

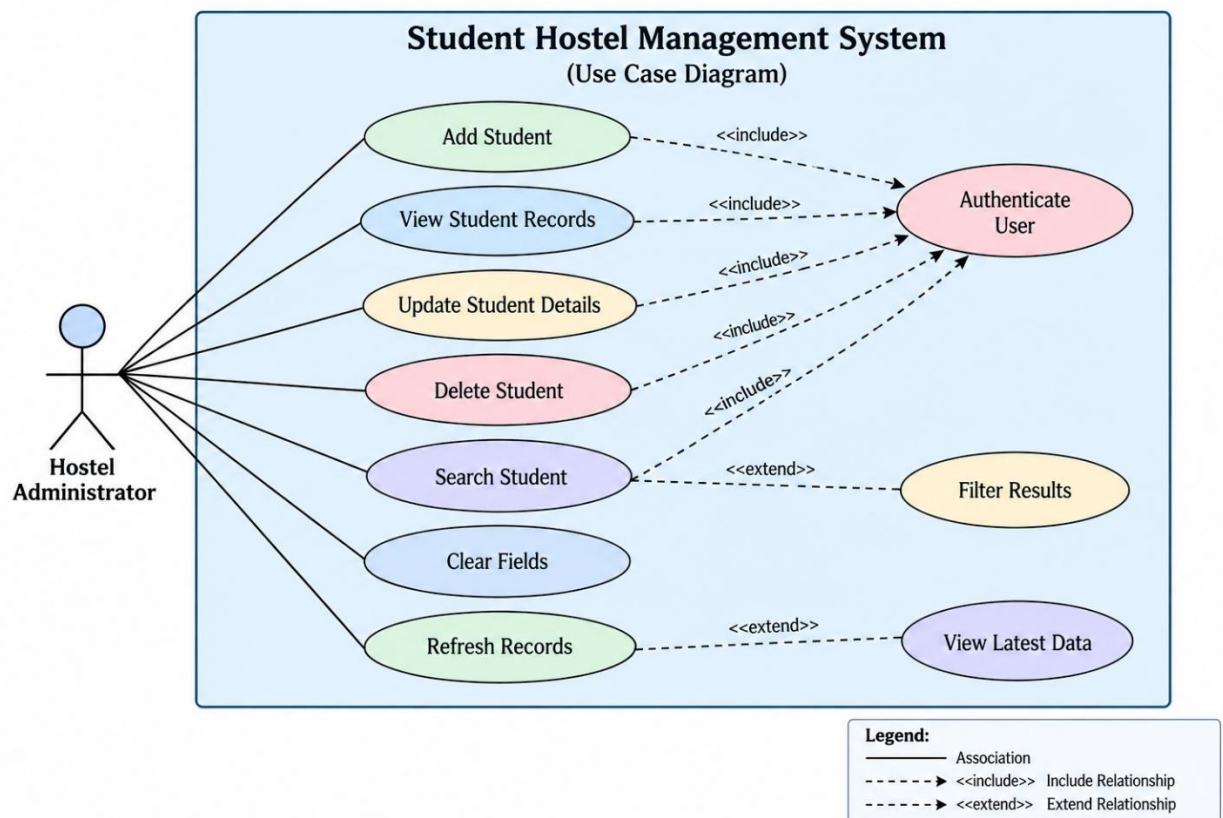
The following design artifacts can be used to explain how the system works and its structure.



9.1 Use Case Diagram

The main actor in the system is the hostel administrator or an authorized user.

The user can perform the actions:



- Add student
- View students
- Update student
- student
- Search student

- **Clear form**
- **Refresh records**

Use Case Representation

9.2 Workflow Diagram

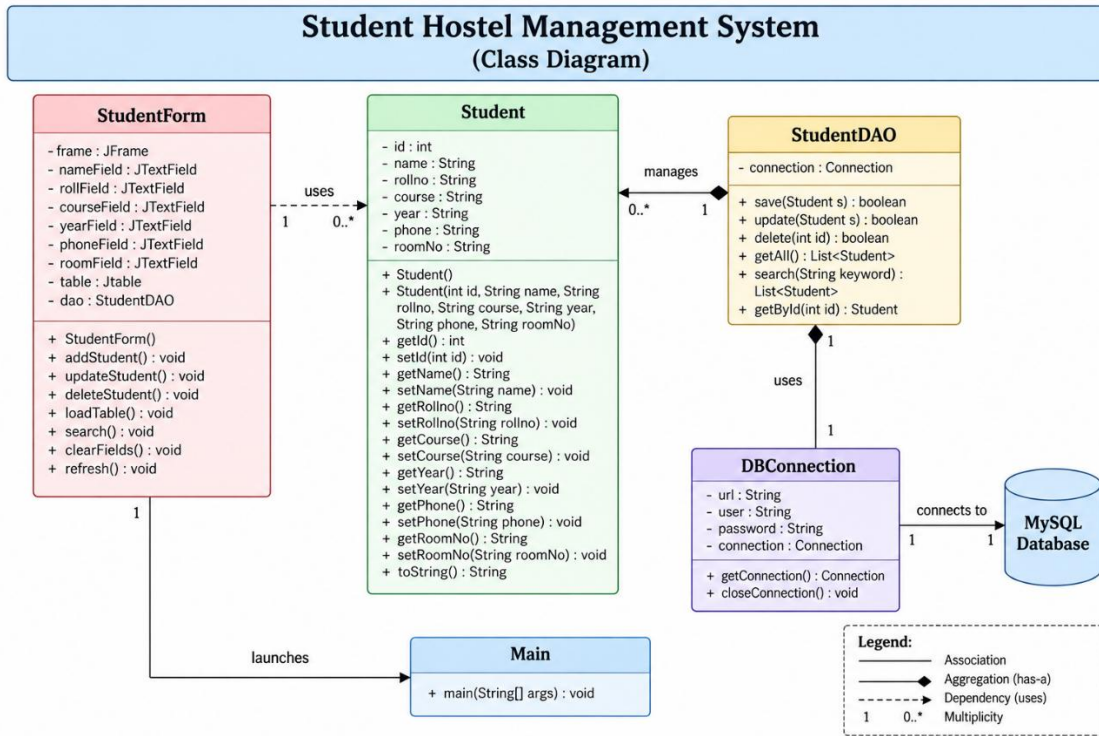
9.3 Sequence Diagram

MySQL

The user enters the details in the form. The form sends the information to the DAO class.

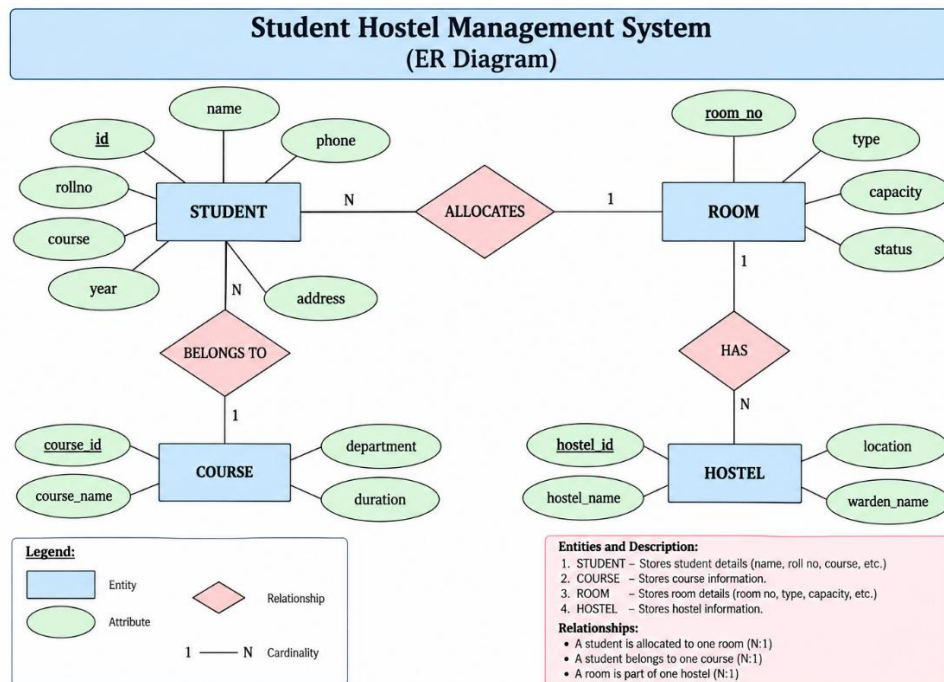
The DAO establishes a database connection. Executes the insert query. After execution the result is displayed to the user.

9.4 Class Diagram



9.5 ER Diagram

The system uses a students table that contains student-related information.



The id field acts as the key that identifies each student record.

10. DATABASE / STORAGE DESIGN

The project uses MySQL as the database management system. Student records are stored in a table called students.

The application performs the following SQL operations:

Insert Operation

Used to add a student record into the database.

Select Operation

Used to retrieve and display all student records.

Update Operation

Used to modify the details of an existing student.

Delete Operation

Used to remove a student record from the database.

The database design is simple and suitable for the project requirements.

11. DESIGN DECISIONS AND RATIONALE

Several design decisions were taken while developing the project.

11.1 Java Swing for GUI

Java Swing was selected because it provides ready-made components such as buttons, labels, text fields, tables and frames. It is suitable for developing a desktop application.

11.2 JDBC for Database Connectivity

JDBC was used because it provides a way to connect Java applications with relational databases such as MySQL.

11.3 MySQL for Data Storage

MySQL was selected because it is widely used supports SQL queries. Is suitable for storing structured student information.

11.4 DAO Pattern

The Data Access Object approach was used to keep database queries, from the user

interface. This makes the code cleaner and easier to maintain.

11.5 Separate Model Class

The Student class is used to represent student information. This helps in organizing the data

11.6 Table-Based Record Display

A JTable was used to display multiple student records in rows and columns. This makes it easier for the user to read and compare details.

11.7 Modular Structure

The project was divided into UI, model, DAO and database connection classes. This decision makes debugging and future modifications easier.

12. IMPLEMENTATION DETAILS

**The project is implemented using Java.
Follows an object-oriented programming
approach.**

Technologies Used

- Java**
- Java Swing**
- JDBC**
- MySQL**
- IntelliJ IDEA / VS Code / Command Line**
- MySQL Connector/J**

Important Project Classes

StudentForm.java

**This class creates the graphical user
interface. I notice that it contains input
fields, buttons, search functionality and the
student records table.**

Student.java

**This class represents the student object.
Contains fields related to student
information.**

StudentDAO.java

**This class contains the database methods
for:**

- . Saving student records**
- . Updating student records**
- . Deleting student records**
- . Fetching all student records**

DBConnection.java

**This class is responsible for creating the
connection between the Java application and
the MySQL database.**

CRUD Operations

The application follows CRUD operations:

- . Create: Add a new student**
- . Read: Display student records**
- . Update: Modify student details**

- . Delete: Remove student details**

These operations form the functionality of the project.

13. SMART LOGIC

Although the project is a management application it includes some useful logic to make record handling easier.

13.1 Search Logic

The application allows the user to search student records using name, roll number or room number. This avoids the need to manually check every row in the table.

13.2 Automatic Table Refresh

After performing database operations the records can be refreshed so that the latest information is displayed.

13.3 Record Selection

The user can select a row from the table. Use its information for updating or deleting the record.

13.4 Centralized Database Operations

All major database operations are handled through the DAO class of directly writing queries inside the user interface.

13.5 Reusable Connection Method

The database connection is handled through a method in DBConnection.java, which avoids repeating the same connection code in multiple places.

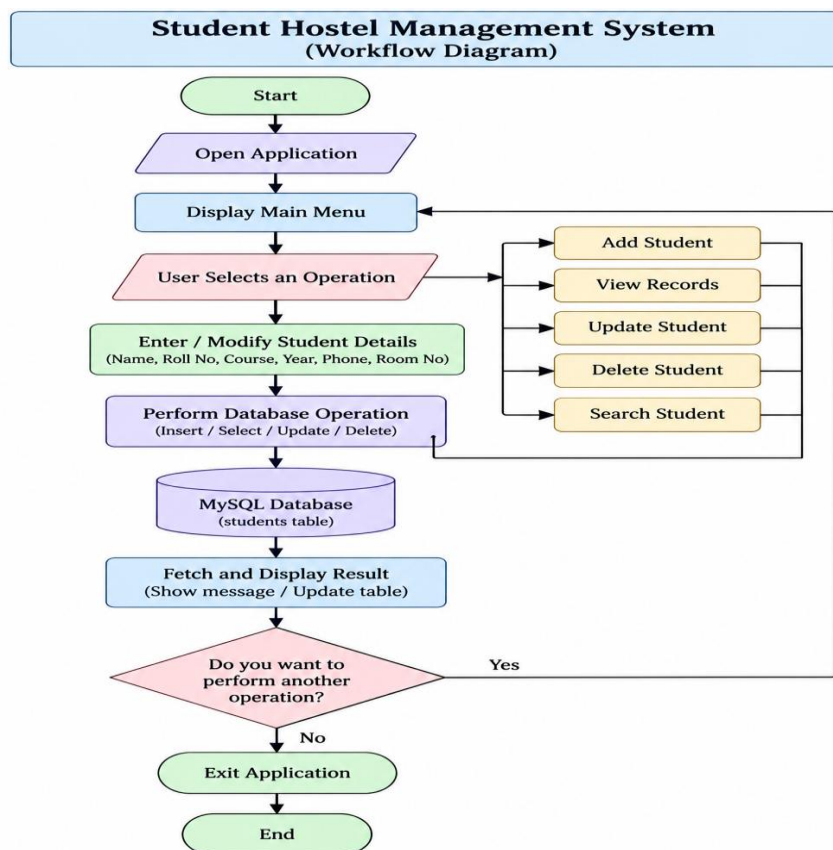
13.6 Input-Based Operations

The system performs operations according to the details entered by the user making it interactive of using fixed values.

The smart logic in this project focuses mainly on reducing work and improving the usability of the application.

14. USER. Workflow

The user interface is designed using Java Swing components. It provides a form through which the user can manage student information.



Main Interface Components

- Name text field
- Roll number text field
- Course text field
- Year text field

- **Phone number text field**
- **Room number text field**
- **Add button**
- **Update button**
- **Delete button**
- **Clear button**
- **Search field**
- **Refresh option**
- **Student records table**

Application Workflow

1. **The user opens the application.**
2. **The student form is displayed.**
3. **The user enters student details.**
4. **The user selects the required operation.**
5. **The application sends the request to the DAO class.**
6. **The DAO executes the SQL query.**

- 7. The database returns the result.**
- 8. The updated information is displayed in the table.**
- 9. The user can continue with another operation. Close the application.**

The interface is kept simple so that even a first-time user can understand the functions.

15. TESTING APPROACH AND RESULTS

Testing was performed to check whether the main functions of the application were working correctly.

15.1 Functional Testing

Each button and operation was checked individually.

Test Case	Expected Result Result
Add student details	Record should be inserted
View student records	Records should appear in table
Update selected student	Details should be updated
Delete selected student	Record should be removed
Search by name	Matching record should be displayed

Test Case	Expected Result	Result
Search by roll number	Relevant record should be found	
Search, by room number	Matching room record should be shown	
Clear button	Input fields should become empty	Passed
Refresh button	Latest records should be displayed	

15.2 Database Testing

Database queries were checked to confirm that:

- Records were inserted correctly which I confirmed.**
- Existing records could be. I verified that this worked.**
- Selected records could be. I tested that each delete succeeded.**

- **Data could be retrieved from the database. I saw the correct output.**

15.3 Interface Testing

The buttons, text fields, table and search area were checked to ensure that they responded properly to user actions.

15.4 Testing Result

The application successfully demonstrated the CRUD operations required for student hostel record management provided the database connection was available and correctly configured.

16. CHALLENGES FACED

During the development of the project some challenges were faced.

16.1 Database Connectivity

Connecting the Java application with MySQL using JDBC required database URL,

username, password, port and connector configuration.

16.2 Handling SQL Queries

Writing SQL queries for insert, update, delete and select operations required careful attention to table and column names.

16.3 Managing GUI Events

Connecting button actions with the database methods was a little challenging in the beginning.

16.4 Updating JTable

Refreshing the table after adding, updating or deleting records required handling of the retrieved data.

16.5 Handling Errors

Database connection errors and incorrect inputs had to be checked during testing.

16.6 Maintaining Separate Classes

Keeping the user interface, model and database code separate required understanding of the project structure.

These challenges helped in understanding how parts of a Java application work together.

17.. Key TAKEAWAYS

This project helped in understanding the application of several programming concepts.

The major learnings include:

- Understanding Java Swing components.**
- Learning how to create a desktop GUI application.**
- Understanding JDBC database connectivity.**
- Writing and executing SQL queries from Java.**
- Implementing CRUD operations.**

- **Understanding the use of DAO classes.**
- **Learning how to organize a project into packages.**
- **Working with JTable to display database records.**
- **Understanding event handling in Java.**
- **Learning the importance of database configuration.**
- **Improving debugging and error-handling skills.**
- **Understanding how software design decisions affect maintainability.**

The project also provided experience in converting a real-world problem into a working software solution.

18. FUTURE ENHANCEMENTS

The current project provides student record management features. Several additional features can be added in the future.

18.1 Login System

A username and password-based login system can be added to restrict access to users.

18.2 Room Allocation

The system can automatically assign rooms based on availability and capacity.

18.3 Fee Management

Hostel fee details, payment status and pending fees can be added.

18.4 Attendance Management

The application can include student attendance records.

18.5 Complaint Management

Students can submit complaints related to rooms, electricity, water or other hostel facilities.

18.6 Report Generation

The system can generate student lists and hostel reports in PDF or Excel format.

18.7 Advanced Search and Filters

More filters such as course, year and room availability can be added.

18.8 Online or Web-Based Version

The desktop application can later be converted into an application for access from multiple devices.

18.9 Backup and Restore

A backup feature can be included to prevent data loss.

18.10 Improved Validation

advanced validation can be added for phone numbers, roll numbers and duplicate records.

20. CONCLUSION

The Student Hostel Management System was built to make it easier to keep student hostel records. The Student Hostel Management System offers functions such as adding, viewing, updating, deleting and searching for student information.

By using Java Swing, JDBC and MySQL the Student Hostel Management System demonstrates how a desktop application can connect to a database and be used for record keeping. The modular structure of the Student Hostel Management System also makes the code easier to read and maintain.

The Student Hostel Management System helped in learning skills with Java GUI, SQL

queries, database connections, event handling and CRUD operations. Although the Student Hostel Management System focuses on student records now it gives a solid base for adding more advanced hostel management features later.

Overall the Student Hostel Management System achieves its goal of creating a clear and organized system for managing hostel student information.

21. REFERENCES

1. Oracle Java Documentation

<https://docs.oracle.com/en/java/>

2. Java Swing Documentation

<https://docs.oracle.com/javase/tutorial/uiswing/>

3. JDBC Documentation

<https://docs.oracle.com/javase/tutorial/jdbc/>

4. MySQL Documentation

<https://dev.mysql.com/doc/>

5. MySQL Connector/J Documentation

<https://dev.mysql.com/doc/connector-j/en/>